

PROJECT REPORT

CAR SALES DATA ANALYZER

A Python-Based Data Analysis and Visualization Project

TITLE PAGE

PROJECT REPORT

ON

CAR SALES DATA ANALYZER

Submitted in partial fulfillment of the requirements for the academic project.

Submitted By

Seru Ganesh

Department: cse

Registration Number:26MIM10066

Submitted To

Faculty Name: Dr. Pricilla dinkar moya

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project report entitled “Car Sales Data Analyzer” is a bonafide work carried out by Seru Ganesh as part of the academic project work.

The project has been developed using Python programming language with the help of Pandas and Matplotlib libraries. It focuses on analyzing automobile sales data, performing data manipulation operations, generating statistical summaries, and visualizing sales trends through graphical representations.

The project work demonstrates the student's understanding of Python programming, data analysis, file handling, and data visualization techniques.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty members and institution for providing me with the opportunity to develop the project titled “Car Sales Data Analyzer.”

I am thankful to my project guide for the valuable guidance, encouragement, and support provided throughout the development of this project.

I would also like to thank my institution for providing the necessary learning environment and resources to complete this project successfully.

This project helped me improve my knowledge of Python programming, data manipulation, statistical analysis, and graphical visualization. I am grateful to everyone who directly or indirectly supported me in completing this project.

ABSTRACT

The Car Sales Data Analyzer is a Python-based application developed to analyze and visualize automobile sales data for different car manufacturers and models over a period of six years, from 2015 to 2020. The main purpose of this project is to simplify the process of examining car sales information stored in CSV files and presenting it in an understandable format.

The application uses the Pandas library for reading, processing, modifying, and analyzing datasets. It provides functionalities such as displaying column names, viewing records, selecting specific car models, adding new records, deleting columns, and generating descriptive statistical summaries.

The project also incorporates Matplotlib to generate line graphs and bar charts representing yearly sales figures. These graphical representations help users observe sales trends and compare the performance of individual car models across different years.

The system follows a menu-driven approach consisting of a Main Menu, Data Analysis Menu, and Graph Menu. It provides a simple console-based interface that allows users to perform different operations without requiring advanced technical knowledge.

This project demonstrates the practical application of Python programming in data analysis and visualization. It can be used as an academic project to understand data processing, statistical operations, and graphical representation of real-world datasets.

Keywords: Python, Car Sales, Data Analysis, Pandas, Matplotlib, CSV, Data Visualization.

TABLE OF CONTENTS

Chapter Title

1	Introduction
2	Problem Statement and Objectives
3	System Analysis
4	System Design

5	System Implementation
6	Testing and Results
7	Conclusion and Future Scope
8	References

CHAPTER 1: INTRODUCTION

1.1 Introduction

The automobile industry is one of the major industries that generates a large amount of sales information every year. Car manufacturers collect sales data to understand customer preferences, analyze market demand, and monitor the performance of different car models.

Sales data generally contains information about manufacturers, vehicle models, and yearly sales figures. Analyzing this information helps users understand how sales change over time and identify patterns in the available dataset.

However, when the information is stored in large CSV files, manually examining the records and comparing sales figures can be difficult and time-consuming.

The Car Sales Data Analyzer is developed to solve this problem by providing a Python-based application that automates basic data analysis and visualization operations.

The system allows users to load car sales information from CSV files and perform various operations through a menu-driven interface. It uses Pandas for data manipulation and Matplotlib for graphical representation.

1.2 Purpose of the Project

The purpose of the project is to develop a simple and efficient application for analyzing car sales data.

The application enables users to examine sales records, calculate statistical values, modify the dataset, and generate graphical representations of sales trends.

It also helps students understand how programming concepts and data analysis libraries can be applied to practical problems.

1.3 Scope of the Project

The project focuses on the analysis of car sales data from 2015 to 2020.

The scope includes:

- Reading sales information from CSV files.

- Displaying and examining dataset records.
- Selecting individual car models and viewing sales figures.
- Adding new car model information.
- Deleting existing model columns.
- Generating descriptive statistical summaries.
- Creating line graphs and bar charts for sales visualization.

The application is designed for educational purposes and basic dataset analysis.

CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Car sales datasets contain large amounts of numerical information related to different automobile manufacturers and models. Analyzing this information manually requires significant time and effort.

Manual analysis may make it difficult to compare sales figures across multiple years, identify sales trends, calculate statistical values, and understand the performance of individual car models.

Furthermore, numerical data presented only in tabular format may not provide a clear understanding of changes in sales over time.

Therefore, a computerized system is required to load, process, analyze, and visualize car sales data efficiently.

The Car Sales Data Analyzer provides a solution by using Python programming, Pandas, and Matplotlib to perform data analysis operations and represent sales information through tables, statistical summaries, and graphs.

2.2 Objectives

The major objectives of the project are:

1. To develop a Python application for analyzing automobile sales data.
2. To read and process CSV files using Pandas.
3. To provide a menu-driven interface for accessing different operations.
4. To display dataset columns and selected records.
5. To retrieve sales information for individual car models.
6. To support adding and deleting car model information.

7. To generate statistical summaries of numerical data.
8. To visualize sales trends using line graphs.
9. To compare yearly sales figures using bar charts.
10. To improve understanding of sales patterns through data visualization.

2.3 Existing System

In the existing manual system, sales information is generally examined using spreadsheets or by manually calculating values from datasets.

This approach can require considerable time when the dataset contains many car models and multiple years of sales records.

The existing approach may involve the following limitations:

- Time-consuming data examination.
- Difficulty in comparing multiple years of sales information.
- Manual effort required for statistical calculations.
- Limited graphical representation.
- Increased possibility of calculation errors.

2.4 Proposed System

The proposed system is a Python-based application that automates basic data analysis operations.

It provides a structured interface for loading CSV datasets, displaying information, modifying records, calculating descriptive statistics, and generating graphs.

The use of Pandas simplifies data manipulation, while Matplotlib provides graphical representations of sales information.

The proposed system reduces manual effort and provides a convenient way to analyze the available dataset.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Functional Requirements

Functional requirements describe the operations that the application must perform.

1. CSV File Loading: The system must allow users to select and load a CSV file containing car sales information.

2. Display Columns: The system must display the names of columns available in the dataset.
3. Display Records: The system must display a specified number of top and bottom rows.
4. Select Car Model: The system must allow users to view sales information for a selected car model.
5. Add New Record: The system must allow users to enter a new car model and its yearly sales figures from 2015 to 2020.
6. Delete Column: The system must allow users to remove a selected car model column.
7. Statistical Summary: The system must calculate and display descriptive statistics for numerical columns.
8. Generate Line Graph: The system must generate a line graph representing the sales trend of a selected car model.
9. Generate Bar Graph: The system must generate a bar chart displaying yearly sales figures.
10. Manufacturer-Based Selection: The system must organize available models by manufacturer to simplify model selection.

3.2 Non-Functional Requirements

The application should provide a simple and understandable menu-driven interface.

It should process the supported CSV dataset efficiently and display the results in a readable format.

The system should be maintainable, allowing additional features and datasets to be incorporated in the future.

It should also provide meaningful error messages when the user enters invalid information or provides an incorrect file path, where appropriate.

3.3 Hardware Requirements

Component Requirement

Processor	Basic modern processor
RAM	4 GB recommended
Storage	Sufficient space for Python and CSV files

System Desktop or laptop

3.4 Software Requirements

Software	Purpose
Python 3	Main programming language
Pandas	Data processing and analysis
Matplotlib	Graph generation
IDLE / VS Code	Development environment
CSV	Dataset storage format

3.5 Feasibility Study

Technical Feasibility: The project can be developed using Python and freely available libraries. It does not require specialized hardware or expensive software.

Economic Feasibility: The project uses free and open-source tools, reducing development and implementation costs.

Operational Feasibility: The menu-driven interface allows users with basic computer knowledge to operate the application.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

The system follows a simple layered approach.

The input layer accepts the CSV file and user selections. The processing layer uses Pandas to manipulate and analyze the dataset. The output layer displays results and generates graphs using Matplotlib.

Input

CSV File and User Input

Data Processing

Python and Pandas

Analysis

Data Selection, Statistics, and Manipulation

Output

Tables, Line Graphs, and Bar Charts

4.2 Data Flow

The application begins by displaying the Main Menu. The user loads a CSV file containing car sales data.

Once the file is loaded, the dataset is stored in a Pandas DataFrame. The user can then select the Data Analysis Menu or the Graph Menu.

The Data Analysis Menu performs operations such as displaying records, selecting columns, adding new models, deleting columns, and generating statistical summaries.

The Graph Menu allows the user to select a manufacturer and car model. The selected model's sales data is then passed to Matplotlib to generate the required graph.

The results are displayed to the user, and the application continues until the user selects the exit option.

4.3 Data Flow Diagram

4.4 Module Description

The application consists of the following modules:

Main Menu Module: Handles the starting interface, CSV file loading, navigation, and exit operation.

Data Analysis Module: Performs data inspection, column selection, record addition, column deletion, and statistical calculations.

Graph Module: Handles manufacturer-based selection and generates line graphs and bar charts.

Data Handling Module: Uses Pandas DataFrames to store and process the loaded dataset.

Visualization Module: Uses Matplotlib to display the selected car model's yearly sales information.

CHAPTER 5: SYSTEM IMPLEMENTATION

5.1 Implementation Overview

The Car Sales Data Analyzer is implemented using Python 3.

The Pandas library is used to read CSV files and perform operations on the dataset. The Matplotlib library is used to create graphical representations of sales information.

The application follows a menu-driven structure, where each menu option performs a particular task.

5.2 CSV File Handling

The system uses the Pandas `read_csv()` function to load sales data from a CSV file.

The loaded data is stored in a DataFrame, which allows the application to access individual columns, display records, and perform statistical operations.

The dataset is expected to contain car model information and sales figures for the years 2015 to 2020.

5.3 Data Manipulation

Pandas provides several functions for modifying and examining the dataset.

The application uses column selection to retrieve sales figures for individual car models. New records can be added by entering model names and sales values.

Similarly, selected columns can be deleted when they are no longer required.

These operations allow the user to manage the dataset during the application session.

5.4 Statistical Analysis

The application uses descriptive statistical functions to summarize numerical sales data.

The summary includes:

- Count: Number of available numerical observations.
- Mean: Average sales value.
- Standard deviation: Measure of variation in sales values.
- Minimum: Lowest sales value.
- Maximum: Highest sales value.
- Quartiles: Values describing the distribution of sales figures.

The statistical summary helps users understand the distribution and variation of the available sales information.

5.5 Data Visualization

The project uses Matplotlib to generate line graphs and bar charts.

The line graph represents the yearly sales trend of a selected car model. It uses years on the horizontal axis and sales figures on the vertical axis.

The bar chart represents the annual sales figures as individual bars, making it easier to compare sales values across years.

These graphs are displayed in separate visualization windows.

5.6 Execution Procedure

The application can be executed using the following steps:

1. Install Python 3 on the computer.
2. Install the required libraries using the command:

```
pip install pandas matplotlib
```

3. Save the Python source code in a .py file.
4. Prepare the CSV dataset with the required sales columns.
5. Run the Python file using IDLE or Visual Studio Code.
6. Select the CSV file through the Main Menu.
7. Choose the required data analysis or graph operation.
8. View the results and exit the application when finished.

CHAPTER 6: TESTING AND RESULTS

6.1 Testing

Testing is performed to verify whether the different modules of the application work according to the specified requirements.

The application can be tested using sample car sales data containing manufacturer names, car models, and annual sales figures.

6.2 Test Cases

Test Case	Expected Result
Load CSV file	Dataset should load into the DataFrame
Display columns	Available column names should be displayed
View top and bottom rows	Requested records should be displayed
Select a car model	Corresponding sales data should be displayed
Add a new model	New model data should be added to the DataFrame
Delete a model	Selected column should be removed
Generate summary	Descriptive statistics should be displayed
Generate line graph	Sales trend should appear in a graph window

Generate bar graph Yearly sales should appear as bars

6.3 Expected Results

After successful execution, the application displays the Main Menu and allows users to load the required dataset.

The Data Analysis Menu provides access to the different data processing and statistical operations.

The Graph Menu displays the available manufacturers and car models for selection. After selecting a model, the application generates a line graph or bar chart representing the sales figures.

The expected results demonstrate the use of Python libraries for organizing, processing, and visualizing automobile sales information.

Note: Actual screenshots and measured results should be inserted after running and testing the application with the project dataset.

6.4 Advantages

- Reduces manual effort in analyzing sales data.
- Provides quick access to car model information.
- Supports basic dataset modification.
- Generates statistical summaries automatically.
- Presents sales information through graphs.
- Uses freely available Python libraries.
- Provides a simple menu-driven interface.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The Car Sales Data Analyzer project demonstrates how Python programming can be used to analyze and visualize automobile sales information.

The application provides functionalities for loading CSV datasets, displaying records, selecting car models, adding and deleting columns, and generating descriptive statistical summaries.

The integration of Pandas and Matplotlib allows the system to process numerical information and represent sales trends through line graphs and bar charts.

The project helped in understanding important programming concepts such as file handling, data manipulation, statistical analysis, and graphical visualization.

The menu-driven structure makes the application suitable for academic demonstrations and basic car sales data analysis.

Overall, the project provides a practical example of converting raw numerical data into meaningful tables, statistical summaries, and visual representations.

7.2 Future Scope

The application can be extended in several ways to improve its functionality.

Future enhancements may include:

1. Developing a graphical user interface using Tkinter.
2. Adding support for sales data from additional years.
3. Incorporating more automobile manufacturers and models.
4. Adding interactive dashboards for sales analysis.
5. Including manufacturer-wise sales comparisons.
6. Implementing database integration using SQLite or MySQL.
7. Adding export functionality for reports and graphs.
8. Incorporating advanced statistical analysis and sales forecasting methods.

These improvements can expand the application into a more comprehensive automobile sales analysis system.

CHAPTER 8: REFERENCES

The following resources can be referred to for understanding and developing the project:

1. Python Official Documentation — <https://docs.python.org/3/>
2. Pandas Documentation — <https://pandas.pydata.org/docs/>
3. Matplotlib Documentation — <https://matplotlib.org/stable/>
4. Python CSV Documentation — <https://docs.python.org/3/library/csv.html>
5. Dataset: Car sales CSV file used for the project.

Report prepared for: Car Sales Data Analyzer

Student Name: Seru Ganesh

Project Language: Python

End of Project Report